

For starters, we'll quickly install the library in a Jupyter Notebook terminal using the following command:

```
pip install qiskit
```

This will install the latest stable version of the library. Qiskit library has four foundational elements: Terra (code foundation and supports circuits and pulses), Aer (Provides simulators and noise models), Ignis (supports noise and error mitigation) and Aqua (supports building algorithms and applications). You can check the version of qiskit and its elements with the following command in the Notebook:

```
import qiskit
qiskit.__qiskit_version__
```

We have used Qiskit version 0.23.0 for performing the coding used in this article.

The basic unit of information in quantum computing is called a 'qubit'. Unlike the classical 'bit', it is capable of handling both $|0\rangle$ and $|1\rangle$ at the same time. The default state of any qubit is $|0\rangle$. To change that, as per the problems, we can perform several computations by creating 'circuits' with the help of 'quantum gates'. We will familiarise ourselves with all the important gates and their functions, as well as their mathematical representations along with the ways to use them to perform actions that are responsible for quantum advantage. There are three types of gates.

1. Single-qubit gates

These gates can be applied on single qubits. Some important gates are as follows.

- a. **Pauli X-Y-Z gates:** The X-gate does bit-flip ($|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$). The Y-gate is responsible for bit as well as phase flip ($|0\rangle$ to $-|1\rangle$). The Z-gate only does phase flip ($|0\rangle$ to $-|0\rangle$). Their corresponding matrices are as follows:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}; \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Following are the lines of code to create a circuit and apply these gates:

```
from qiskit import QuantumCircuit
# Performing an X Gate
qc = QuantumCircuit(1) # initializing
a quantum circuit with 1 qubit
qc.x(0) # applying x gate on q0
qc.draw(output='mpl') # drawing the
circuit to visualize
```

To check execute the circuit on the *statevector* simulator provided by Qiskit Aer:

```
from qiskit import Aer, execute
from qiskit.visualization import plot_
bloch_multivector
backend = Aer.get_backend('statevector_
simulator')
result = execute (qc, backend).result().
get_statevector(qc, decimals=3)
plot_bloch_multivector(result)
```

Figure 1 shows the outputs of above two code snippets and the working of X gate.

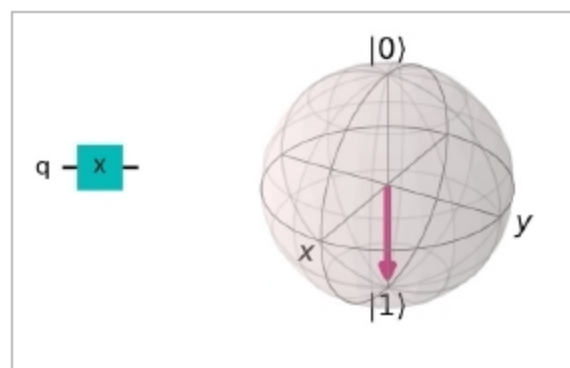


Figure 1: Showing the circuit with X gate and its statevector plotted on Bloch sphere

The X gate flips the state from $[1 + 0.j, 0 + 0.j]$ to $[0 + 0.j, 1 + 0.j]$. Similarly, if we apply Y gate and check the output, we get: $[0. - 0.j, 0. + 1.j]$, i.e., bit as well as phase flip, and Z gate gives us $[1. + 0.j, -0. + 0.j]$, i.e., just the phase flip.

- b. **Hadamard gate:** A Hadamard gate represents a rotation of π about the axis that is in the middle of the

X-axis and Z-axis. It maps the basis state $|0\rangle$ to $(|0\rangle + |1\rangle) / \sqrt{2}$, which means that a measurement will have equal probabilities of being $|1\rangle$ ($|0\rangle + |1\rangle$) or $|0\rangle$, creating a 'superposition' of states. This state is also written as $|+\rangle$.

The corresponding matrix of the Hadamard gate is:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

This can be checked using the following lines of code:

```
qc = QuantumCircuit(1)
qc.h(0)
qc.draw(output='mpl')
```

To visualise the output obtained by executing the above circuit, use the following code:

```
backend = Aer.get_backend('statevector_
simulator')
result = execute(qc, backend).result().
get_statevector(qc, decimals=3)
print(result)
plot_bloch_multivector(result)
```

The print statement gives us the *statevector* - $[0.707 + 0.j, 0.707 + 0.j]$. It can be noticed that the two numbers are complex numbers, and have real and imaginary parts. All the quantum states are complex numbers. Here the statevector has zero as imaginary part (0.j) and 0.707 as real part. This means we get the state: $\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$, after applying the Hadamard gate.

Figure 2 gives the output of the above code.

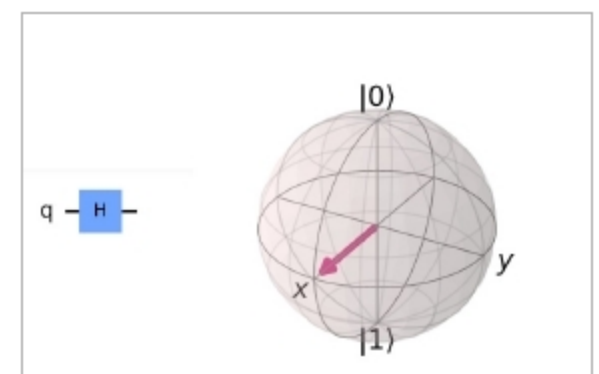


Figure 2: The circuit with Hadamard gate and statevector output plotted on Bloch sphere